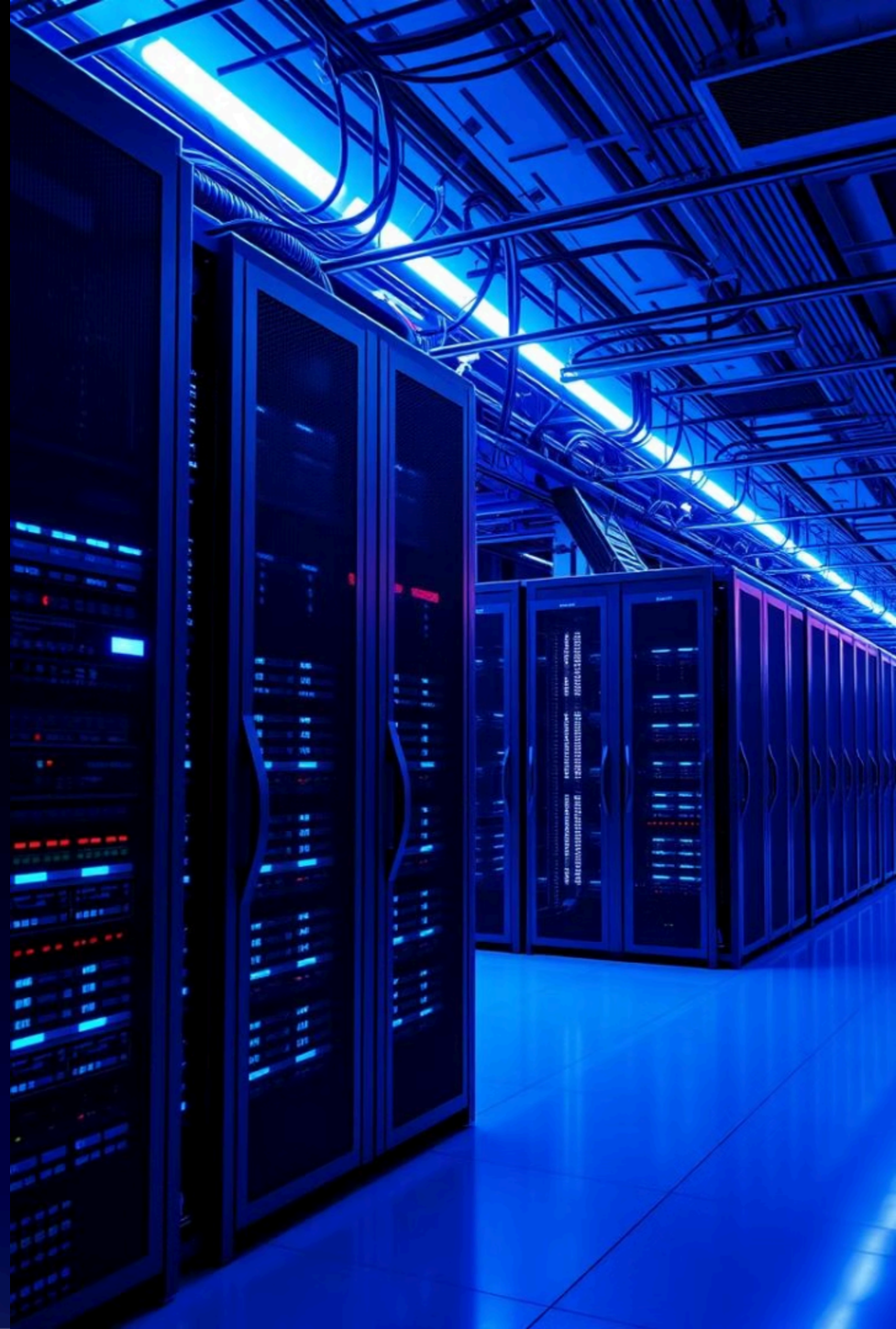
Azure Microservices CI/CD Architecture

Infrastructure as Code, GitHub Actions, and Azure Container Apps

- Santiago Barraza
- Dylan Bermúdez



Project Overview

Architecture

Multi-repository microservices with API Gateway, Health Check and External Configuration Store patterns.

Infrastructure

Managed using Terraform for declarative resource provisioning.

CI/CD

Implemented via GitHub Actions automating build and deployment.

Deployment

Microservices run on Virtual Machine.

Methodology

Kanban

- Work items are tracked on a Kanban board.
- Tasks move through stages: To Do → In Progress → In Review → Done.
- Supports continuous delivery and flexible prioritization.
- Helps the team visualize progress and adapt quickly to changes.

Branching Strategy

GitHub Flow

- main branch:
  - Contains the stable, production-ready version of the code.
  - All deployments to production (or production-like environments) are based on the main branch.
  - Only code that has been tested, validated, and approved via Pull Requests (PRs) is merged into main.
- feature/ branches:
  - Used to develop new features, fixes, or improvements.
  - Branches are created from main with names like feature/create-dockerfile, feature/add-healthcheck, etc.
  - Each feature/ branch:
  - Is tested independently, including running and validating CI/CD pipelines.
  - Undergoes a code review via Pull Request before merging.

Infrastructure Management (IaC)

Provisioned Resources

- **Azure Container Apps**: Managed service to deploy and scale containerized apps without managing infrastructure.
- Azure Container Registry (ACR): Private registry to store and manage Docker container images securely.

Automation

GitHub Actions automate Terraform deployment workflows.

- **Infrastructure Repo**: Contains Terraform scripts for provisioning Azure resources.
- **Microservices Repos**: Each microservice has its own repo for building and deploying Docker images to Azure Container Apps.

Additional Resources

- Service Principal / App Registration
  - Allow Terraform to change infrastructure
  - Allow GitHub for pushing container images to ACR
- Storage Account with Blob Storage for Terraform state (tfstate)

CI/CD Workflows

1

Infrastructure Pipeline

- Trigger: Push to main branch
- Action: Terraform applies infrastructure changes

2

Microservices Pipelines

- Trigger: Push to main branch
- Action: Build Docker images, push to ACR

Microservices Overview

Frontend

Vue.js and Node.js. Includes de frontend and a reverse proxy to send requests to zipkin or the api gateway.

API Gateway

Node.js with Express. Redirects frontend requests to auth-api or todos-api.

Auth API

Golang service for authentication that generates JWT token.

Users API

Java Spring Boot backend that stores and validates users.

Todos API

Node.js with Express. Manage users requests to get, create or delete tasks.

Redis

Stores data of user's tasks

Log Message Processor

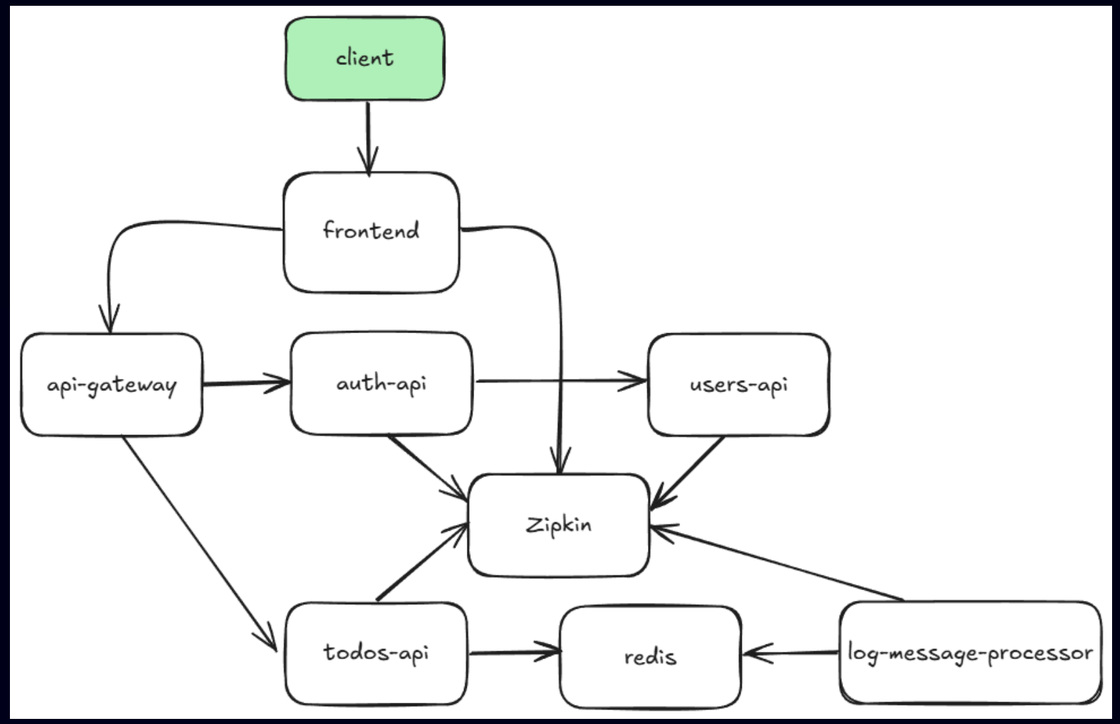
Python service logging messages of what is happening in Redis

Zipkin

Distributed tracing tool, receiving traces from microservices and showing their interaction

Service Communication Flow

- Frontend forwards traffic via API Gateway
  - API Gateway routes to Auth or Todos API
    - Auth API validates users with Users API
      - Todos API stores tasks in Redis cache
    - Log Processor reads events from Redis
  - Zipkin collects traces from all services except Redis and API Gateway



Architectural Patterns Applied

API Gateway Pattern

Central entry for requests with unified authentication and routing.

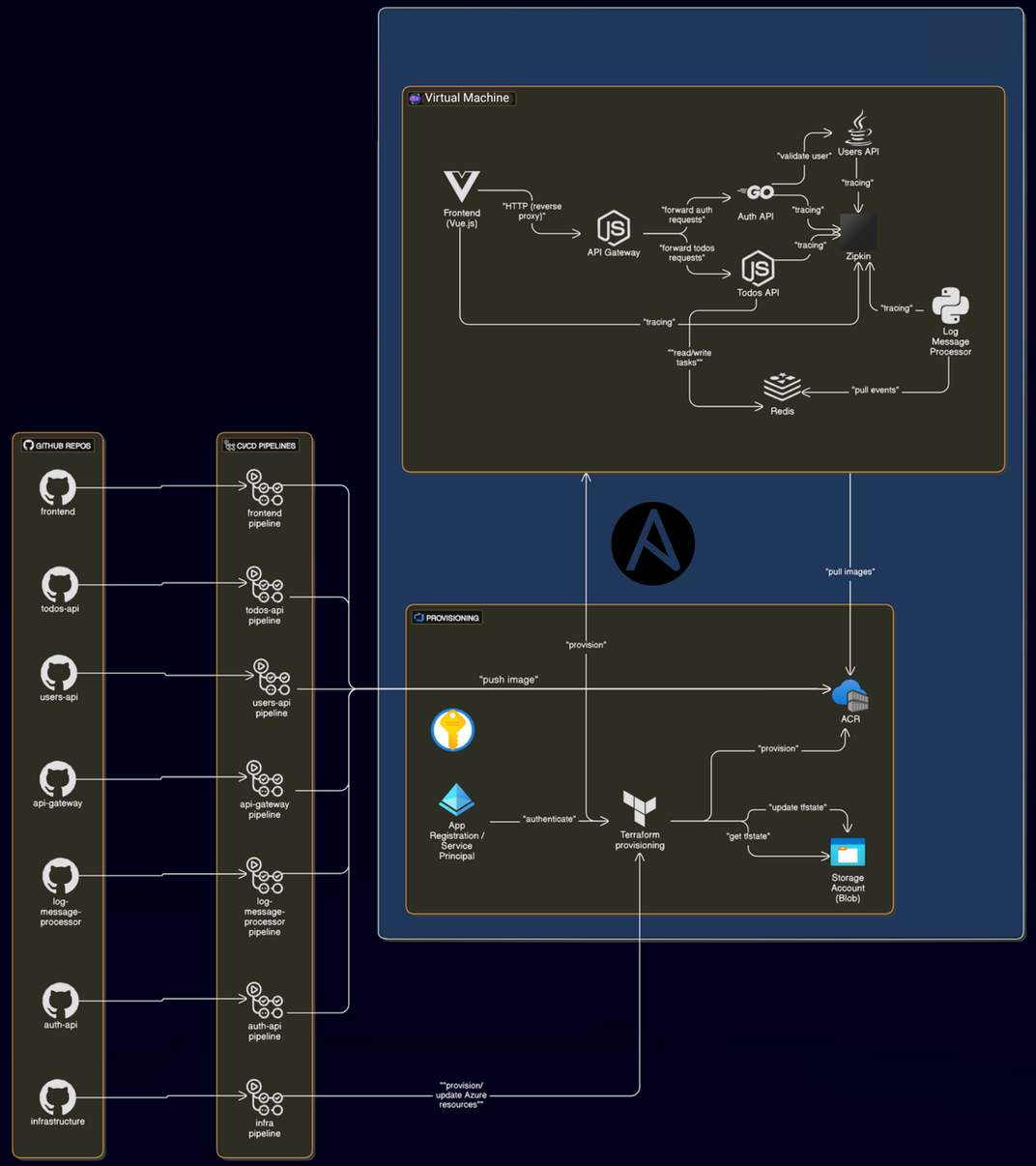
Health Check Pattern

Services expose /health endpoint monitored by Azure Container Apps.

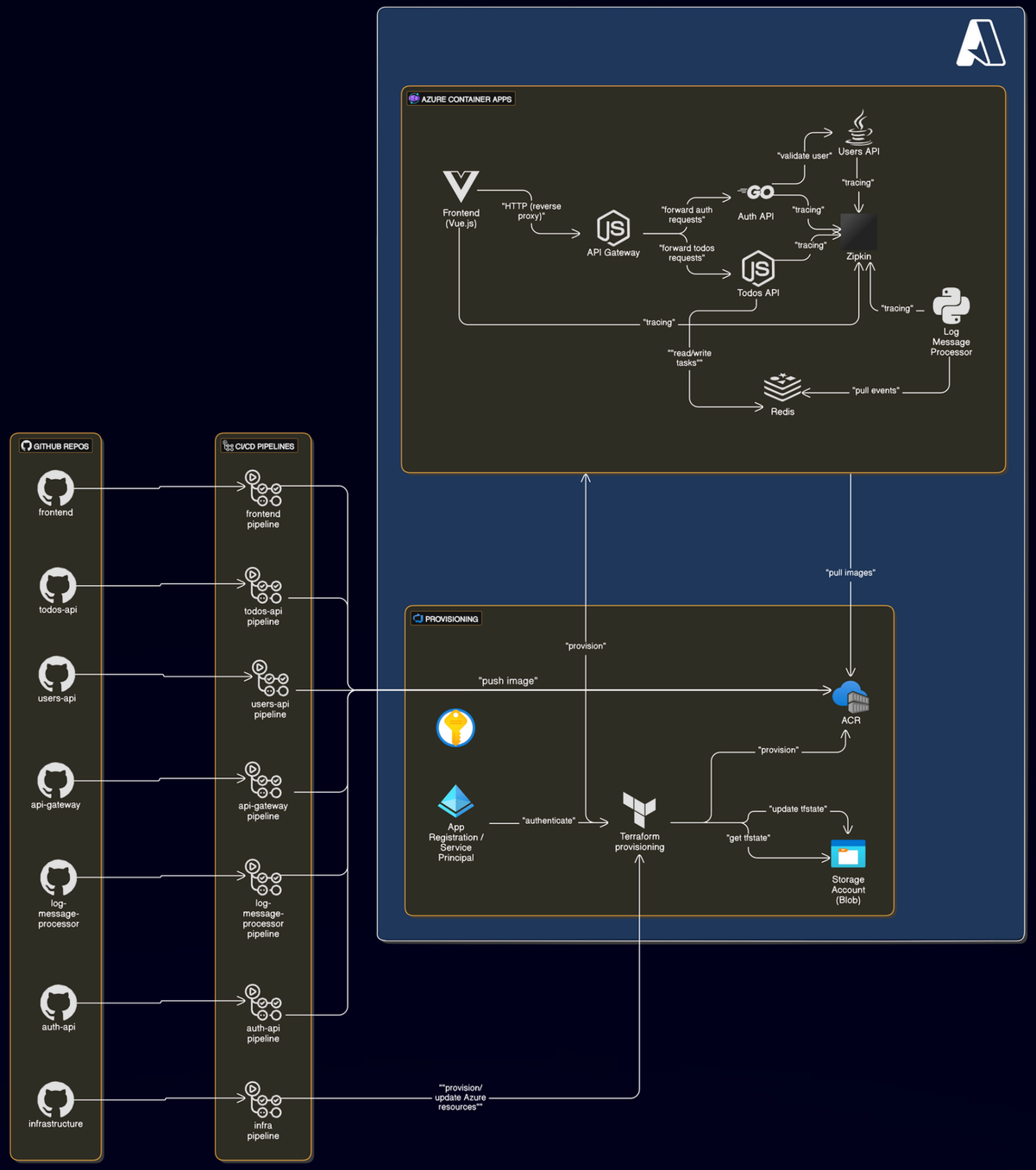
External Configuration Store

Azure key vault stores VM IP address so the code pipelines can access it

System Architecture



- **CI/CD Pipeline**:
  GitHub Actions automates the entire process.
  - On a push to the main branch, the infrastructure pipeline runs Terraform to manage and update the cloud resources.
  - Microservices pipelines build Docker images and push them to Azure Container Registry (ACR). These images are automatically deployed to Azure Container Apps.
- **Terraform Infrastructure**:
  Infrastructure is provisioned and updated using Terraform.
  - Resources managed include **Azure Container Apps**, **Azure Container Registry** for storing Docker images, and **Azure Storage Account** for managing the Terraform state (tfstate).
- **Container Deployment**:
  Docker images are stored in ACR and deployed to **Azure Container Apps**, allowing for scalable and isolated microservices execution.



Best Solution

Summary

End-to-End Automation

GitHub Actions and Terraform ensure full pipeline automation.

Azure Container Apps Deployment

Scalable platform tailored for container-based microservices.

High Reliability

Health monitoring and auto-recovery boost availability.

Modular Architecture

Clear separation of concerns via API Gateway pattern.

Thank You